

מועד ב - פתרון שאלה 1

a. `SELECT MAX(hire_date) AS newest_hire_date
FROM employees;`

b. `SELECT
d1.dept_no AS dept1,
d2.dept_no AS dept2
FROM dept_emp AS d1
JOIN dept_emp AS d2
ON d1.emp_no = d2.emp_no
AND d1.dept_no < d2.dept_no
GROUP BY
d1.dept_no,
d2.dept_no
HAVING COUNT(DISTINCT d1.emp_no) > 3;`

השאלתה אינה צריכה להיות רפלקסיבית, משום שאין משמעות להשוואת מחלקה לעצמה. אנחנו מחפשים עובדים שעבדו בשתי מחלקות שונות. הקשר עצמו סימטרי, אם עובד עבד במחלקה A ובמחלקה B אז כמובן שהוא עבד גם ב-A וגם ב-B. עם זאת אין צורך להחזיר אותו זוג פעמיים.

c. `SELECT DISTINCT
t1.emp_no,
t1.title AS title_1,
t2.title AS title_2,
t1.from_date AS title_1_from,
t1.to_date AS title_1_to,
t2.from_date AS title_2_from,
t2.to_date AS title_2_to
FROM titles AS t1
JOIN titles AS t2
ON t1.emp_no = t2.emp_no
AND t1.title < t2.title
AND t1.from_date < t2.to_date
AND t2.from_date < t1.to_date;`

בחרתי להחזיר את `emp_no` כדי לזהות את העובד, את שני התפקידים כדי להראות שמדובר בשני `title` שונים, ואת תאריכי ההתחלה והסיום של שני התפקידים כדי שניתן יהיה לראות ולהוכיח שהתקופות חופפות. אין צורך להחזיר את שם העובד, משום ש-`emp_no` הוא המפתח שמזהה את העובד באופן חד-חד ערכי. התנאי:

`t1.title < t2.title`

גם מבטיח שמדובר בשני תפקידים שונים וגם מונע החזרה כפולה של אותו זוג תפקידים בסדר הפוך

פתרון נוסף בהנחה ששתי התקופות זהות:

```
SELECT
    ,emp_no
    ,from_date
    to_date
FROM titles
GROUP BY
    ,emp_no
    ,from_date
    To_date
HAVING COUNT(DISTINCT title) >= 2
```

```
d. SELECT
    s1.emp_no,
    s1.from_date AS decrease_date
FROM salaries AS s1
JOIN salaries AS s2
    ON s1.emp_no = s2.emp_no
    AND s1.from_date = s2.to_date
WHERE s1.salary < s2.salary;
```

פתרון נוסף עם windows functions:

```
SELECT
    emp_no,
    from_date AS decrease_date
FROM (
    SELECT
        emp_no,
        from_date,
        salary,
        LAG(salary) OVER (
```

```

        PARTITION BY emp_no
        ORDER BY from_date
    ) AS previous_salary
FROM salaries
) AS s
WHERE salary < previous_salary;

```

```

e. CREATE VIEW employees_per_department AS
SELECT
    d.dept_no,
    COUNT(DISTINCT de.emp_no) AS employee_count
FROM departments AS d
LEFT JOIN dept_emp AS de
    ON d.dept_no = de.dept_no
GROUP BY d.dept_no;

```

```

SELECT
    employee_count,
    COUNT(*) AS department_count,
    COUNT(*) * 1.0 /
        (SELECT COUNT(*) FROM employees_per_department) AS probability
FROM employees_per_department
GROUP BY employee_count
ORDER BY employee_count;

```

f.

סיבה 1:

יתכן שיש טבלאות אחרות שמכילות foreign key ללקוח שאותו אנחנו מנסים למחוק.

אם ה foreign key אינו מאפשר מחיקת הרשומה, בסיס הנתונים ימנע את המחיקה כדי לא לפגוע בתקינות הנתונים.

סיבה 2:

name אינו בהכרח שדה ייחודי.

ייתכן שבסיס הנתונים פועל ב**Safe Update Mode**. במצב זה פעולת **DELETE** עשויה להיחסם כאשר תנאי ה-**WHERE** אינו משתמש במפתח. אם **name** אינו ייחודי, הוא אינו יכול לשמש כמפתח ייחודי, ולכן מחיקה לפי **name = Dan** עשויה להיחסם.

עצם זה שיכולים להיות כמה לקוחות בשם **Dan** לא גורם לשאילתה להיכשל. אם המחיקה מותרת, כל הרשומות שמתאימות לתנאי יימחקו.

סיבה 3:

ייתכן שהטבלה **customers** אינה קיימת.

במקרה כזה בסיס הנתונים לא יוכל לבצע את השאילתה.

g. CREATE VIEW department_distribution AS

SELECT

employee_count,

COUNT(*) AS department_count

FROM employees_per_department

GROUP BY employee_count;

SELECT

employee_count,

SUM(department_count) OVER (

ORDER BY employee_count

) * 1.0 /

(SELECT COUNT(*) FROM departments) AS cumulative_probability

FROM department_distribution

ORDER BY employee_count;